SUGGESTION

overing Over the Water

PERSONAL PROJECT - 2026

Watchlist

A personal film library rebuilt from a Letterboxd export.

Letterboxd is a social platform. This needed to be a personal one.

- Letterboxd is optimised for logging, rating, and being seen — not for private use.
- Your favourites list exists, but inside someone else's interface and taxonomy.
- The actual need: at 11pm you know you want something *slow and devastating* — not a genre filter.
- The gap is between taste (specific, embodied) and retrieval (assumes you know what to ask for).

> *"I know what I like. I don't always know what I want to watch."*

Three things Letterboxd doesn't do.

- Surface films by **emotional register**, not just genre.
- Feel like a **personal collection**, not a social database.
- Give the films **room** — not reduce them to star ratings and activity feeds.

The question was never "how do we display films?" It was "what is the right relationship with this collection?"

The data was a starting point, not the product.

A Letterboxd export is a flat CSV — title, URI, date added, rating. That's it.

Every entry was enriched via TMDB: backdrop, poster, synopsis, director, cast, streaming. The schema also carries a personal layer — **notes** and **tags** that feed back into search.

That transforms a spreadsheet into a library.

prisma/schema.prisma — Data model

```
model Film {
  id              Int      @id @default(autoincrement())
  tmdbId          Int      @unique
  title           String
  year            Int
  runtime         Int?
  genres          String   // JSON; enriched from TMDB
  posterPath      String?
  backdropPath    String?
  synopsis        String?
  director        String?
  cast            String?

  // Personal layer — these feed back into search
  notes           String?  // observations, creative references, free text
  tags            String?  // mood/creative labels e.g. '["melancholy","wide angles"]'

  letterboxdUri   String?  @unique   // stable re-sync key from export
  letterboxdRating Float?
  watched         Boolean  @default(false)
  playlists       PlaylistFilm[]
}
```

04 - DESIGN DIRECTION

A library, not a watchlist.

TYPOGRAPHY

Xanh Mono for display — monospaced, editorial, precise. Not consumer. Work Sans 300 for body — light and unobtrusive.

PALETTE & LAYOUT

Near-black, neutrals. Films as objects, not content. Full-bleed backdrops at low opacity — atmosphere before information.

TONE OF VOICE

"Tonight's suggestion" not "Featured." *"A moment."* not "Loading..." The copy is curatorial throughout.



05 - THE LIBRARY

Browse.

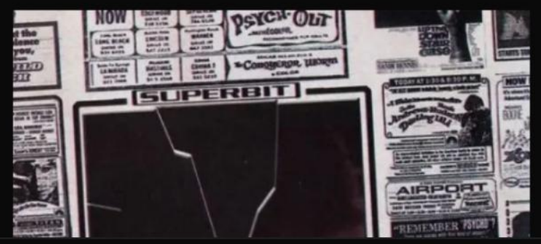
Filter by genre, decade, runtime, streaming. "Tonight's suggestion" — a random pick, resets on refresh. Serendipity as a feature, not an afterthought.

Watched indicator: a single pixel-thin line at the top of the poster card. Minimal — enough.

_1que

Runtime Service Status









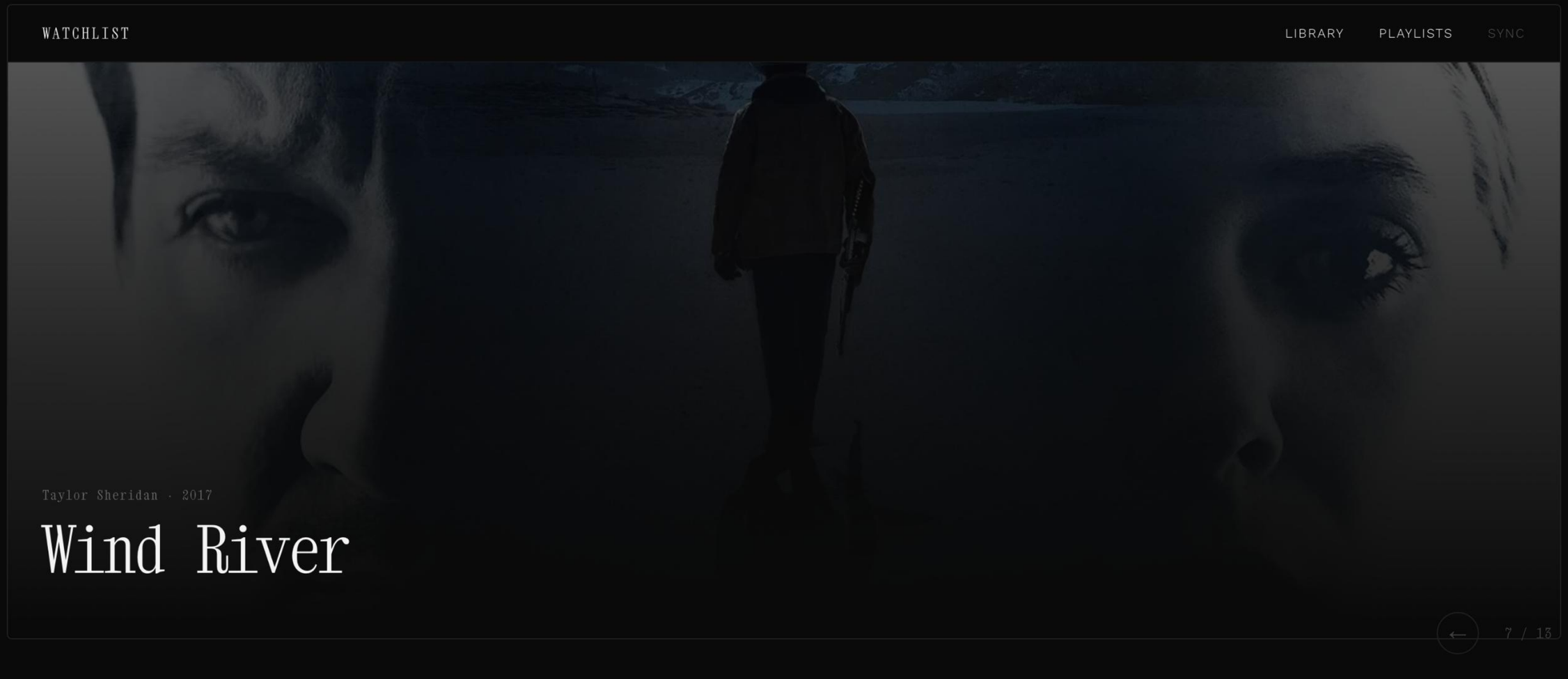
← 5 / 13 →

06 - FILM PAGE

Atmosphere before information.

Full-bleed backdrop at low opacity. Personal notes field — free text, for your own references and observations. Custom tags: mood labels, visual references, anything.

The annotations feed back into search. The more you write, the sharper the library gets.



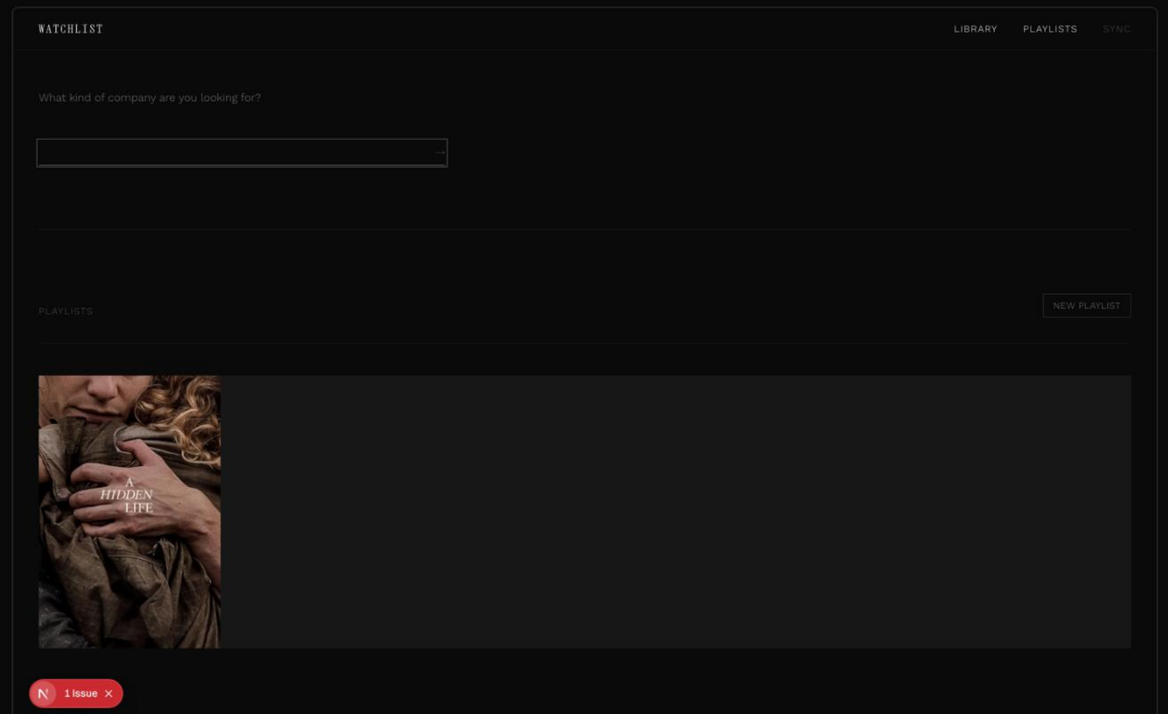
07 - ITERATION

The search box was wrong.

First version: a search box. Problem: search is transactional — it assumes you know what you want.

The insight: most of the time you know how you *feel*, not what you want. So the interface needed to meet you there.

Decision: make it conversational. Two steps — mood, then sharpener. The follow-up questions pattern-match emotional vocabulary and ask one precise question back.



No AI. Pure vocabulary.

In-memory keyword scoring, runs in under 5ms. The key decision: a hand-built table maps emotional language to genres. "Melancholy" → Drama. "Noir" → Crime, Thriller, Mystery.

Why not an LLM? Latency felt wrong for something meant to feel immediate. Being explicit about the vocabulary is also more honest — you can see exactly what the system knows about your taste.

lib/explore.js — Mood → Genre vocabulary

```
// Maps emotional language to TMDB genre names.
// The gap this bridges: users say "melancholy" — data says "Drama".
const MOOD_GENRES = {
  sad:          ['Drama', 'Romance'],
  melancholy:   ['Drama'],
  grief:        ['Drama'],
  lonely:       ['Drama'],
  isolation:    ['Drama', 'Thriller'],
  tender:       ['Drama', 'Romance'],
  wistful:      ['Drama', 'Romance'],
  dark:         ['Thriller', 'Crime', 'Horror', 'Mystery'],
  bleak:        ['Drama', 'Crime'],
  existential:  ['Drama'],
  quiet:        ['Drama'],
  slow:         ['Drama'],
  meditative:   ['Drama', 'Documentary'],
  atmospheric:  ['Drama', 'Horror', 'Thriller'],
  dreamlike:    ['Fantasy', 'Drama'],
  haunting:     ['Drama', 'Horror'],
  noir:         ['Crime', 'Thriller', 'Mystery'],
  // ... 80+ terms mapping mood to genre
}
```

lib/explore.js — Field weights

```
// Weighted scoring: intentional annotations beat raw metadata.
// title (6) > genre (5) > tags (4) > notes (3) > director (3) > cast (2) > synopsis (1)
function scoreFilm(film, terms) {
  let score = 0
  for (const term of terms) {
    if (title.includes(term))    score += 6
    if (genres.includes(term))   score += 5
    if (tags.includes(term))     score += 4   // your own tags, highest signal
    if (notes.includes(term))    score += 3   // your own notes
    if (director.includes(term)) score += 3
    if (cast.includes(term))     score += 2
    if (synopsis.includes(term)) score += 1

    // Mood → genre boost: "melancholy" → Drama films score +4
    for (const g of (MOOD_GENRES[term] ?? [])) {
      if (genres.includes(g.toLowerCase())) score += 4
    }
  }
  return score
}
```

Editorial voice written as code.

The opening questions rotate. The follow-ups don't offer a menu — they ask one more precise question. The copy was written before the code.

> *"Something to sit inside, or to understand from a distance?"*

> *"Slow cinema proper, or just something that doesn't push?"*

components/Companion.js — Editorial voice in code

```
// The curator doesn't always open the same way.
const OPENINGS = [
  "What are you after tonight?",
  "What does tonight feel like?",
  "Where are you, emotionally speaking?",
  "What kind of company are you looking for?",
  "Something to sit inside, or somewhere to go?",
  "What does the evening call for?",
  "What are you not in the mood for? That's sometimes easier.",
]

// Each follow-up sharpens the request — one question, not a menu.
const FOLLOWUPS = [
  {
    pattern: /\b(sad|grief|loss|mourn|heartbreak|broken|devastat)\b/i,
    question: "Something to sit inside, or to understand from a distance?"
  },
  {
    pattern: /\b(slow|quiet|still|contemplat|meditat|patient|unhurried)\b/i,
    question: "Slow cinema proper, or just something that doesn't push?"
  },
  {
    pattern: /\b(beauti|visual|cinemat|gorgeous|strik|image|colour|light)\b/i,
    question: "Style as the whole point, or in service of something else?"
  },
]
```

Decisions made.

DECISION	FIRST VERSION	WHY IT CHANGED
Hero framing	Featured film	"Tonight's suggestion" — editorial framing changes the relationship with the content
Discovery	Search box	The Companion — search assumes prior knowledge you often don't have at 11pm
Mood → genre	Direct genre filter only	MOOD_GENRES vocabulary — bridges how you think to how data is tagged
Search engine	Explored Claude API	Latency + cost felt wrong; hand-built vocabulary is more immediate and honest
Watched marker	Badge / icon overlay	1px top border — present enough to register, absent enough not to dominate

What this taught.

ON DESIGN

- "Reskinning" was the wrong frame. This was reimagining the *relationship* with the collection.
- Tone of voice is an interface decision. Copy written before UI, not after.
- Personal data transformed by design becomes a different kind of object.

ON PRODUCT

- The best interface matches how you actually think, not how the data is structured.
- No AI doesn't mean no intelligence — the vocabulary table *is* the intelligence, it's just explicit.
- A personal tool should feel made, not configured.

11 - NEXT

Where it goes.

- Direct sync from Letterboxd when their API opens
- Richer tagging — pull themes from notes into structured tags automatically
- Mobile-first viewing mode — the library you actually use on the sofa